

Python Programming

Unit 06 – Lecture 02 Notes

NumPy Functions (Stats, Sorting, Searching)

Tofik Ali

February 20, 2026

Contents

1	Lecture Overview	1
2	Core Concepts	2
2.1	Generating Arrays	2
2.2	Reshape	2
2.3	Statistics and Axis	2
2.4	Sorting and Searching	2
2.5	Dot Product and Matrix Multiplication	3
3	Demo Walkthrough	3
4	Interactive Checkpoints (with Solutions)	3
5	Practice Exercises (with Solutions)	3
6	Exit Question (with Solution)	4

1 Lecture Overview

This lecture introduces frequently used NumPy functions for:

- array generation,
- reshaping,
- statistics,
- sorting and searching,
- and basic linear algebra.

2 Core Concepts

2.1 Generating Arrays

Useful functions:

- `np.zeros`, `np.ones`
- `np.empty` (uninitialized values)
- `np.arange`, `np.linspace`
- random arrays: `np.random.randint`, `np.random.random`

```
import numpy as np

a = np.zeros((2, 2))
b = np.ones((2, 2))
c = np.arange(0, 10, 2)
d = np.linspace(0, 1, 5)
r = np.random.randint(1, 10, size=(3, 3))
```

2.2 Reshape

`reshape` changes the array shape without changing data:

```
x = np.arange(1, 10) # size = 9
m = x.reshape((3, 3)) # 3x3 matrix
```

2.3 Statistics and Axis

Common statistics:

- `sum`, `mean`, `std`, `min`, `max`

Axis meaning for 2D array:

- `axis=0`: operate down the rows (column-wise result)
- `axis=1`: operate across columns (row-wise result)

```
print(m.sum(axis=0)) # column sums
print(m.sum(axis=1)) # row sums
```

2.4 Sorting and Searching

Sorting:

```
arr = np.array([3, 1, 2])
print(np.sort(arr)) # returns new sorted array
arr.sort() # sorts in-place
```

Searching:

```
arr = np.array([10, 50, 20, 80])
idx = np.argmax(arr)
print(idx, arr[idx])
```

2.5 Dot Product and Matrix Multiplication

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a.dot(b)) # 1*4 + 2*5 + 3*6

M = np.arange(1, 10).reshape((3, 3))
print(M @ M)
```

3 Demo Walkthrough

File: demo/numpy_functions_demo.py

Observe:

- generating random arrays,
- reshaping and computing stats along axes,
- sorting and searching for max/2nd max,
- dot product and matrix multiplication.

4 Interactive Checkpoints (with Solutions)

Checkpoint 1 Solution

Question: meaning of `axis=0`?

Answer: compute column-wise results (down the rows).

Checkpoint 2 Solution

Question: difference between `np.sort(a)` and `a.sort()`?

Answer:

- `np.sort(a)` returns a new sorted array.
- `a.sort()` sorts the array in-place (mutates `a`).

5 Practice Exercises (with Solutions)

Exercise 1: Row and Column Sums

Task: Create a 3x3 array and compute row sums and column sums.

Solution:

```
import numpy as np
m = np.arange(1, 10).reshape((3, 3))
print("Row sums:", m.sum(axis=1))
print("Col sums:", m.sum(axis=0))
```

Exercise 2: Find Second Maximum

Task: Find second maximum element of an array.

Solution (simple):

```
import numpy as np
arr = np.array([10, 6, 8, 90, 12, 56])
u = np.unique(arr)
u.sort()
print("Second max =", u[-2])
```

6 Exit Question (with Solution)

Question: one function to generate sequence?

Answer: np.arange (or np.linspace)

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Quick Links

[Core Concepts](#) [Demo](#) [Interactive](#) [Summary](#)

Agenda

- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Use important NumPy functions: `reshape`, `sum`, `mean`, `std`
- Generate arrays using `random`, `zeros`, `ones`, `empty`
- Sort and search arrays
- Compute dot products and matrix multiplication

Reshaping Arrays

```
a = np.arange(1, 10)
m = a.reshape((3, 3))
```

- Total elements must match (size stays same)

Statistics

- `sum`, `mean`, `std`, `min`, `max`
- Use `axis=` for row/column-wise operations

```
print(m.sum(axis=0)) # column sums
print(m.sum(axis=1)) # row sums
```

Sorting and Searching

- `np.sort`, `np.argsort`
- `np.max`, `np.argmax`
- `np.where(condition)`

Dot Product / Matrix Multiplication

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a.dot(b))

M = np.arange(1, 10).reshape((3, 3))
print(M @ M)
```

Demo: NumPy Functions

- File: `demo/numpy_functions_demo.py`
- Demonstrates:
 - random arrays
 - reshape and axis-based sums
 - sorting and searching
 - dot product and matrix multiplication

Checkpoint 1

Question: What does `axis=0` mean in `sum(axis=0)` for a 2D array?

Checkpoint 2

Question: What is the difference between `np.sort(a)` and `a.sort()`?

Think-Pair-Share

Discuss:

- When should you use `reshape` in data analysis?

Key Takeaways

- `reshape` changes shape without changing data
- Axis-based operations compute row/column stats
- NumPy supports fast sorting/searching and linear algebra operations

Exit Question

Name one NumPy function to generate a sequence of numbers.